Lucas Swanson
6/1/2026
Software Engineering Methods

Homework 9

Part A: Argoproject - [argoproj/argoproj: Common project repo for all Argo Projects]

1. Using Argo Project as a baseline for a continuous delivery software server architecture, we can easily see how they have implemented both Canary, and Blue Green deployment strategies for the user to select when utilizing their software for customizable rollout strategies listed in the repositories readme and supporting repository used for handling the rollouts.
   - Documentation:
   Canary - Argo Rollouts - Kubernetes Progressive Delivery Controller
   BlueGreen - Argo Rollouts - Kubernetes Progressive Delivery Controller
   This shows how Argo handles rollouts and how flexible it can be, allowing the controller to tweak exactly how each rollout is handled either through Canary or BlueGreen strategies.

   This is also easily shown in their agro-rollouts support repository: argoproj/argo-rollouts: Progressive Delivery for Kubernetes





```
            - containerPort: 8080
    strategy:
      blueGreen:
        activeService: rollout-bluegreen-active
        previewService: rollout-bluegreen-preview
        autoPromotionEnabled: false
---
kind: Service
apiVersion: v1
metadata:
  name: rollout-bluegreen-active
spec:
  selector:
    app: rollout-bluegreen
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080

---
kind: Service
apiVersion: v1
metadata:
  name: rollout-bluegreen-preview
spec:
  selector:
    app: rollout-bluegreen
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

2. For this system, I need to make one major assumption: that the basis and premise would be that the development team would need to have access to a good amount of customization and how the server operates to take advantage of the built-in deployment strategies, which rules out Serverless platforms, and some of Platform as a Service.
   Scenario:
   a. On-Premises:
      Team Context: 10 (ish) people, with high levels of deployment experience. Would be required to be On-Call as the website is intended to be operational 24/7. And as hardware is on-site, someone would have to be capable of being present.
      Usage profile: Steady, uptime 100% or during working hours (would affect team size), and growth would be slow due to requiring additional hardware, and geographic accessibility is limited to hardware location.
      Risk/Compliance Profile: Security would have to be held in house and robust. Further increasing the amount of on-site employment required for all safety regulations that must be performed and upheld within internal company implementation.
      Operational Constraints: Budget would be extreme. Requiring teams and

physical locations in each geographic region desired to support, with hardware updates as physical units break or wear down and constant maintenance, on-site security and much more.

b. Infrastructure as a Service:
   Team Context: 5 (ish) people, with a moderate to high level of deployment experience. On-Call status would still be required, but only software resolution would be required, so on-site is not a must (nor normal).
   Usage profile: Can grow faster than On-Premises strategies because if additional bandwidth is required, simply renting additional space is an option, but not automatic.
   Risk/Compliance Profile: This requires very little security as the servers are held and maintained by a third party. Although data security would still be the job of the implementers, connections and physical security are held by the rented company.
   Operational Constraints: If expansion is required, it is typically simply paid for to the rented company. Expansion is easy as the rented company (such as AWS) already has physical locations and servers maintained in many countries. One constraint would be that almost all server adjustments, customizations, and utilizations would be managed by the implementors through server API's.

Part B:

Prompt used:
"You are a senior DevOps engineer helping a client select an appropriate CI/CD pipeline.

Analyze the following scenario and create a table detailing the comparison between GitHub Actions, and Jenkins.

Scenario:

Trailcart is a mid-sized, e-commerce system with:

1. A browser-based storefront

2. An API Back end,

3. Checkout workflows

4. Inventory synchronization

5. Periodic promotional campaigns

The team will use this pipeline to release frequently and experience traffic spikes during sales events.

The team has limited DevOps capacity, and needs the system to be maintainable, secure, rollback-friendly, and have an budget deployment workflow."

GenAI Output:

Analyze the following scenario and create a table detailing the comparison between GitHub Actions, and Jenkins.
Scenario:
Trailcart is a mid-sized, e-commerce system with:
1. A browser-based storefront
2. An API Back end,
3. Checkout workflows
4. Inventory synchronization
5. Periodic promotional campaigns

Show more

Below is a senior-level CI/CD comparison tailored specifically to **Trailcart's architecture and constraints** (mid-sized e-commerce, frequent releases, traffic spikes, limited DevOps capacity, strong need for rollback safety and security).

CI/CD Platform Comparison: GitHub Actions vs Jenkins (Trailcart Scenario)

Category	GitHub Actions	Jenkins
Operational Overhead	Fully managed SaaS (or minimal self-hosted runners). Very low maintenance burden. Ideal for small DevOps teams.	Fully self-hosted. Requires ongoing management (updates, plugin maintenance, scaling agents, backups). High overhead.
Setup Speed	Fast (native integration with GitHub repo, minimal configuration).	Slower (server setup, plugin configuration, agent provisioning).
Scalability for Traffic Spikes (Sales Events)	Scales automatically via hosted runners or elastic self-hosted runners (cloud-integrated). Good fit for burst workloads like promotions.	Scales, but requires manual or pre-configured build agents. Risk of bottlenecks if not architected properly.
CI/CD Pipeline Flexibility	Strong for most modern workflows (YAML-based pipelines, reusable actions). Slightly constrained for very complex orchestration.	Extremely flexible. Highly customizable pipelines via Groovy DSL and plugins. Best for complex, non-standard workflows.
Security Posture	Strong by default (managed infrastructure, automatic patching, fine-grained secrets, OIDC support). Lower attack surface.	Security depends entirely on team. Requires constant patching and plugin vulnerability management. Higher risk if not maintained properly.
Maintenance Burden	Very low. GitHub manages infrastructure, updates, and scaling.	High. Requires DevOps resources for upgrades, plugin conflicts, and server health.
Rollback Support	Strong support via workflow versioning + integration with deployment tools (Kubernetes, AWS, etc.). Easy to implement automated rollback strategies.	Fully capable but requires manual configuration (scripts, plugins, custom logic). More fragile if not engineered carefully.
Deployment Strategy Support (Blue/Green, Canary)	Excellent when paired with modern cloud tools (ArgoCD, AWS CodeDeploy, Kubernetes). Native integration patterns exist.	Possible but requires plugin assembly and custom scripting. More effort to standardize across teams.
Cost Model	Free tier + usage-based billing for runners. Predictable for mid-sized workloads but can increase during high CI usage.	Free software, but infrastructure cost (servers, VMs, storage, maintenance labor) can be significant and unpredictable.
Plugin Ecosystem / Extensibility	Moderate but growing ecosystem (Actions marketplace). Quality is generally consistent.	Massive plugin ecosystem, but quality varies widely; plugin conflicts are common.
Integration with GitHub Repo	Native (tight coupling with PRs, issues, environments, approvals). Excellent developer experience.	Requires external integration setup (webhooks, credentials, SCM configuration).
Auditability & Compliance	Strong built-in logs, environment protections, approvals, and traceability.	Also strong, but requires manual configuration of audit pipelines and log retention policies.
Best Fit for Trailcart	Ideal for lean teams needing secure, scalable, low-maintenance CI/CD with frequent deployments and seasonal traffic spikes.	Better suited for large enterprises with dedicated DevOps/SRE teams and highly specialized pipeline requirements.

Critique:
I think this matches the Trailcart scenario quite well, as many of the questions from a client's perspective are already answered without me having to specify what I was looking for. Things such as scalability during traffic spikes, maintenance burden on the client, setup speed, operational overhead required, cost models, and more. These are some of the things I would think about if I were attempting to set up a system quickly and safely. While this table is quite extensive, a few key things are missing. Primarily, the ability to see into any errors or debugging capacity, which is where Jenkins would shine, due to the sheer customizability within the tool, as it can be locally hosted. Another thing to consider is the developer experience required to support each choice, which is where GitHub Actions would shine, as it is a built-in tool for GitHub, where as Jenkins is an internal tool that would be customized by the implementing team. As someone consulting a new or young company, I would warn them about Jenkins requiring some adept DevOps staff. Lastly, I would likely recommend GitHub actions as it seems, based on the scenario, that scalability, maintainability, and security are all big items for the client, and those are where GitHub Actions shines.

Part C:

1. Considering the use of Docker for the back-end deployment of SpeedScore, it may be an alright candidate. Firstly the assumption here is that the use of Docker is only for deployment of SpeedScore, and seeing as SpeedScore was originally intended as a solo project created by one person, this is not a bad idea. Some pro's would be that the mutability of Docker would be quite consistent. Docker is locally operated and would not be subject to much change. But a con for this would be that this would be a huge burden if SpeedScore were ever either to be worked on or fixed by multiple people, or if SpeedScore were to be deployed to multiple servers or regions, as this is a pretty manual process. Any automation would have to be done with some other tool, so scalability is quite low. But as it is a manual process, it would interrupt all usage of the server while updates are happening.
2. Pulumi is a much more robust and automatic infrastructure option for implementing the back-end deployment of SpeedScore. This would allow for much more unified server updates rolled out across multiple servers in various areas as it uses methods like Kubernetes which uses a phased deploiyment strategy. This reduces risk to currently operating servers as well as can be applied automatically. This also would allow for users to continue to upload and view their previous results as servers are slowly updated one by one, allowing for seamless usage of the

application while updates are happening with no interruptions (slight bandwidth changes while updates are happening). This would also be very scalable and would not modify the infrastructure state as this is most common with IaaS or PaaS.